

ANGULAR SIGNAL FORMS

DISCLAIMER

! IMPORTANT: Signal Forms are [experimental](#). The API may change in future releases. Avoid using experimental APIs in production applications without understanding the risks.

SIGNAL FORMS

Signal Forms wurden mit Angular 21 als experimentelle API bereitgestellt

- Form-State basiert auf Signals
- Automatische Synchronisation UI ↔ Modell
- Vollständig Typsicher
- Zentralisierte und Schemabasierte Validierung

FORM-MODELL SIGNAL

Ausgangspunkt für eine Signal Form ist ein Signal

```
interface LoginData {  
  email: string;  
  password: string;  
}  
  
@Component()  
export class Login {  
  loginModel = signal<LoginData>({  
    email: '',  
    password: '',  
  });  
}
```

FIELDTREE

Mit der form()-Funktion wird ein FieldTree auf Basis des Signals erstellt:

```
@Component()  
export class Login {  
  loginModel = signal<LoginData>({  
    email: '',  
    password: '',  
  });  
  
  loginForm = form(loginModel)  
}
```

Über den so erstellten FieldTree kann nun auf die einzelnen Felder zugegriffen werden:

```
loginForm.email;  
loginForm.password;
```

TEMPLATE BINDINGS

Die Formularfelder können mithilfe der [formField]-Direktive an HTML-Inputs gebunden werden

```
<input type="email" [formField]="loginForm.email" />  
<input type="password" [formField]="loginForm.password" />  
<button [disabled]="!loginForm.valid()" (click)="login()">Login</button>
```

FELD STATUS UND VALUE

Um auf den Status sowie den Value einzelner Felder zuzugreifen, muss man sie wie eine Funktion aufrufen

```
loginForm.email();
```

So erhalten wir ein Objekt vom Typ FieldState. Dieses Objekt enthält Signals für den Value und die verschiedenen Status des Feldes:

```
loginForm.email().value();  
loginForm.email().valid();  
loginForm.email().invalid();  
loginForm.email().touched();  
loginForm.email().dirty();  
loginForm.email().pending();  
loginForm.email().disabled();  
loginForm.email().readonly();  
loginForm.email().hidden();
```

VALUE UND STATUS SETZEN (FELD)

Der Value einzelner Felder kann über die set-Funktion geändert werden:

```
loginForm.email().value.set('test@sn-group.de');
```

Die Funktionen markAsDirty und markAsTouched können verwendet werden um einzelne Felder als Dirty/Touched zu kennzeichnen:

```
loginForm.email().markAsDirty();  
loginForm.email().markAsTouched();
```

Mithilfe der reset-Funktion können der Dirty und Touched Status zurückgesetzt werden:

```
loginForm.email().reset();
```

Der Value bleibt dabei unberührt, falls dieser auf einen Wert zurückgesetzt werden soll, müssen wir diesen mit übergeben:

```
loginForm.email().reset('');
```

VALUE UND STATUS SETZEN (FORM)

Der Value der Kompletten Form kann ebenso über die set-Funktion geändert werden:

```
loginForm().value.set({ email: 'test@sn-group.de', password: 'StarkesPasswort' });
```

Die Funktionen markAsDirty und markAsTouched können verwendet werden um alle Felder eines Formulars als Dirty/Touched zu kennzeichnen:

```
loginForm().markAsDirty();  
loginForm().markAsTouched();
```

Mithilfe der reset-Funktion können der Dirty und Touched Status des kompletten Formulars zurückgesetzt werden:

```
loginForm().reset();
```

Der Value bleibt auch hier unberührt und muss explizit zurückgesetzt werden:

```
loginForm().reset({email: '', password: ''});
```

SCHEMA-FUNKTION - VALIDATOREN

Der form-Funktion kann an zweiter Stelle eine Schema-Funktion übergeben werden. Die Funktion bekommt einen SchemaPathTree übergeben. Dieser kann verwendet werden um Validationsregeln zu konfigurieren:

```
const form = form(loginModel, (schemaPath) => {  
  required(schemaPath.email);  
  email(schemaPath.email);  
});
```

Angular bietet, wie auch schon für ReactiveForms, eine ganze Reihe an Standardvalidatoren an, u.a.:

- required
- email
- min/max
- minLength/maxLength
- pattern
- ...

SCHEMA-FUNKTION - VALIDATOREN

Es ist möglich den Validatoren eine Fehlermeldung für die spätere Anzeige im Template mitzugeben:

```
const form = form(loginModel, (schemaPath) => {  
  required(schemaPath.email, {message: 'Email ist ein Pflichtfeld!!'});  
  email(schemaPath.email, , {message: 'Bitte geben sie eine valide Mail-Adresse ein!'});  
});
```

SCHEMA-FUNKTION - VALIDATOREN

Bedingte Validationen können mithilfe der when-Funktion umgesetzt werden:

```
registrationForm = form(this.registrationModel, (schemaPath) => {  
  required(schemaPath.numberOfChildren, {  
    message: 'Sie müssen angeben wie viele Kinder Sie haben!',  
    when: ({valueOf}) => valueOf(schemaPath.hasChildren),  
  });  
});
```

SCHEMA-FUNKTION - VALIDATOREN

Validierungsfehler können über das errors-Signal gelesen werden. Sie enthalten folgende Felder:

- kind: Die fehlgeschlagene Validationsregel ('required', 'email', ...)
- message: Optional eine konfigurierte Fehlermeldung für den jeweiligen Fehler

```
<input type="email" [formField]="loginForm.email" />
@for (error of registrationForm.email().errors(); track error) {
  <p>{{ error.message }}</p>
}
<input type="password" [formField]="loginForm.password" />
@for (error of registrationForm.password().errors(); track error) {
  @if(error.kind === 'required') {
    <p>{{ 'Das Feld ist Pflicht!' }}</p>
  } @else {
    <p>{{ 'Bitte geben Sie einen korrekten Wert ein!' }}</p>
  }
}
```

SCHEMA-FUNKTION - VALIDATOREN

Über das errorSummary-Signal können wir alle Fehler eines Feldes/Formulars und all ihrer nachfolger auslesen:

```
...
@for (error of registrationForm.errorSummary(); track error) {
  <p>{{ error.message }}</p>
}
...
```

SCHEMA-FUNKTION - EIGENE VALIDATOREN

Mithilfe der validate-Funktion können auch eigene Validatoren geschrieben werden:

```
urlForm = form(this.urlModel, (schemaPath) => {
  validate(schemaPath.website, ({value}) => {
    if (!value().startsWith('https://')) {
      return {
        kind: 'https',
        message: 'URL muss mit https:// starten!',
      };
    }
    return null;
  });
});
```

Die validate-Funktion bekommt den Pfad des zu validierenden Felds übergeben. Im Fehlerfall muss eine Error-Objekt zurückgegeben werden, ansonsten null oder undefined.

SCHEMA-FUNKTION - EIGENE VALIDATOREN

Validatoren sollten im Sinne der Wiederverwendbarkeit (und Lesbarkeit) in einzelne Funktionen ausgelagert werden:

```
export function url(path: SchemaPathTree<string>, options?: {message?: string}) {
  validate(path, ({value}) => {
    if (!value().startsWith('https://')) {
      return {
        kind: 'https',
        message: options?.message || 'URL muss mit https:// starten!',
      };
    }
    return null;
  });
}

urlForm = form(this.urlModel, (schemaPath) => {
  url(schemaPath.website);
});
```

SCHEMA-FUNKTION - EIGENE CROSS-FIELD VALIDATOREN

Validatoren können auch die Werte von mehreren Feldern prüfen. Dazu kann der komplette SchemaPath übergeben werden.

```
passwordsMatch(
  path: SchemaPathTree<{ password: string; confirmPassword: string }>,
  options?: { message?: string },
) {
  validate(path, (ctx) => {
    const passwordValue = ctx.fieldTree.password().value();
    const confirmPasswordValue = ctx.fieldTree.confirmPassword().value();
    // const passwordValue = ctx.valueOf(path.password);
    // const confirmPasswordValue = ctx.valueOf(path.confirmPassword);
    if (passwordValue && confirmPasswordValue && passwordValue !== confirmPasswordValue) {
      return { kind: 'passwordsMatch', message: options?.message };
    } else {
      return null;
    }
  });
}
```

```
registrationForm = form(this.registrationModel, (schemaPath) => {
  passwordsMatch(schemaPath);
});
```

Der Fehler wird in diesem Fall an die Form gehängt:

```
@for (error of registrationForm().errors(); track error) {
  <p>{{ error.message }}</p>
}
```

SCHEMA-FUNKTION - EIGENE CROSS-FIELD VALIDATOREN

Es ist auch möglich von einem Feld aus auf Schwesterfelder zuzugreifen.

```
passwordsMatch(
  path: SchemaPathTree<{ password: string; confirmPassword: string }>,
  options?: { message?: string },
) {
  validate(path.confirmPassword, (ctx) => {
    const passwordValue = ctx.value();
    const confirmPasswordValue = ctx.valueOf(path.password);
    if (passwordValue && confirmPasswordValue && passwordValue !== confirmPasswordValue) {
      return { kind: 'passwordsMatch', message: options?.message };
    } else {
      return null;
    }
  });
}
```

```
registrationForm = form(this.registrationModel, (schemaPath) => {
  passwordsMatch(schemaPath);
});
```

In diesem Fall wird der Fehler an das confirmPassword-Feld gehen.

SCHEMA-FUNKTION - EIGENE TREE VALIDATOREN

Außerdem kann eine Validation auf Formular-Ebene den Fehler explizit an ein oder mehrere Felder hängen. Dazu muss die `validateTree`-Funktion verwendet werden:

```
passwordsMatch(
  path: SchemaPathTree<{ password: string; confirmPassword: string }>,
  options?: { message?: string },
) {
  validateTree(path, (ctx) => {
    const passwordValue = ctx.fieldTree.password().value();
    const confirmPasswordValue = ctx.fieldTree.confirmPassword().value();
    if (passwordValue && confirmPasswordValue && passwordValue !== confirmPasswordValue) {
      return {
        kind: 'passwordsMatch',
        field: ctx.fieldTree.password, // alternativ: [ctx.fieldTree.password, ctx.fieldTree.confirmPassword]
        message: options?.message,
      };
    } else {
      return null;
    }
  });
}

registrationForm = form(this.registrationModel, (schemaPath) => {
  passwordsMatch(schemaPath.confirmPassword);
});
```

In diesem Fall wird der Fehler an das password Feld gehen (oder an das password und das confirmPassword Feld wenn beide angegeben wurden)

SCHEMA-FUNKTION - ASYNCHRONE VALIDATOREN

Asynchrone Validatoren können mithilfe der `validateHttp`-Funktion erstellt werden

```
emailAlreadyUsed(path: SchemaPathTree<string>, options?: {message?: string}) {
  validateHttp(path, {
    request: (ctx) => ({
      url: `${environment.url}/auth/checkEmail`,
      params: {
        email: ctx.value(),
      },
    }),
    onSuccess: (emailAlreadyUsed: boolean) => {
      if (emailAlreadyUsed) {
        return {
          kind: 'emailAlreadyUsed',
          message: options?.message || 'validation.emailAlreadyUsed',
        };
      }
      return null;
    },
    onError: (error) => {
      console.error('api error validating email', error);
      return {
        kind: 'api-failed',
      };
    },
  });
}

registrationForm = form(this.registrationModel, (schemaPath) => {
  emailAlreadyUsed(schemaPath.email);
});
```

SCHEMA-FUNKTION - WIEDERVERWENDBARE SCHEMA

Mithilfe der `apply`-Funktion können ausgelagerte Schema-Definitionen hinzugefügt werden:

```
@Component(...)
export class PaymentInformationForm {
  paymentInformationForm = input.required<FieldTree<PaymentInformation>>();
}

export const PaymentInformationSchema = schema<PaymentInformation>((path) => {
  required(path.iban);
  iban(path.iban);
});
```

Bei einem flachen Formular wird dafür das komplette SchemaPath an die `apply`-Funktion übergeben:

```
registerForm = form(this.registerModel, (schemaPath) => {
  ...
  apply(schemaPath, PaymentInformationSchema);
});

<sn-payment-information-form [paymentInformationForm]="registerForm" />
```

SCHEMA-FUNKTION - WIEDERVERWENDBARE SCHEMA

Analog dazu können auch verschachtelte Formulare erstellt werden

Bei einem flachen Formular wird dafür das komplette SchemaPath an die apply-Funktion übergeben:

```
registerModel = signal<RegistrationDto>({
  ...
  paymentInformation: {
    iban: ''
  }
});

registerForm = form(this.registerModel, (schemaPath) => {
  ...
  apply(schemaPath.paymentInformation, PaymentInformationSchema);
});

<sn-payment-information-form [paymentInformationForm]="registerForm.paymentInformation" />
```

SCHEMA-FUNKTION - WIEDERVERWENDBARE SCHEMA

Mit der applyWhen-Funktion ist es möglich Schema-Definitionen nur unter bestimmten Bedingungen hinzuzufügen

```
registerForm = form(this.registerModel, (schemaPath) => {
  ...
  applyWhen(schemaPath, (ctx) => ctx.valueOf(path.hasBankAccount), PaymentInformationSchema);
});
```

Die applyWhenValue-Funktion kann dabei tipparbeit sparen wenn für die Bedingung nur auf Values der Form zugegriffen werden muss.

```
registerForm = form(this.registerModel, (schemaPath) => {
  ...
  applyWhenValue(schemaPath, (register) => register.hasBankAccount, PaymentInformationSchema);
});
```

SCHEMA-FUNKTION - WIEDERVERWENDBARE FORMULARTEILE FORMARRAYS

FormArrays können mithilfe der applyEach-Funktion hinzugefügt werden

```
settingsModel = signal<User>({
  name: '',
  addresses: [],
});

settingsForm = form(this.settingsModel, (schemaPath) => {
  required(schemaPath.name);
  applyEach(schemaPath.addresses, addressSchema);
});

export const addressSchema = schema<Address>((path) => {
  required(path.city);
  required(path.streetNr);
  required(path.zip);
});

export const initAddress: Address = {
  streetNr: '',
  zip: '',
  city: '',
};
```


SCHEMA-FUNKTION - WIEDERVERWENDBARE FORMULARTEILE FORMARRAYS

Im Template können wir nun über die Adressen iterieren:

```
<div class="addresses">
  @for (
    address of settingsForm.addresses;
    track address;
    let last = $last;
    let count = $count
  ) {
    <div class="address">
      <sn-address-form-sf [addressForm]="address" />
      @if (last) {
        <button mat-mini-fab (click)="addAddress()" color="primary">
          <mat-icon>add</mat-icon>
        </button>
      }
    </div>
  }
</div>
```

Neue Adressen können über die Form hinzugefügt werden

```
addAddress(): void {
  this.settingsForm?.addresses().value.update((addresses) => [...addresses, { ...initAddress }]);
}
```

SCHEMA-FUNKTION - DEBOUNCE

Das Update des Values eines Feldes kann mithilfe der debounce-Funktion verzögert werden:

```
registrationForm = form(this.registrationModel, (schemaPath) => {
  emailAlreadyUsed(schemaPath.email);
  debounce(schemaPath.email, 400);
});
```

Das Form-Model erhält in diesem Beispiel erst einen neuen Wert für das email-Feld wenn:

- der Nutzer 400ms lang keine Eingabe gemacht hat
- der Nutzer das Feld als touched markiert (indem er das Feld verlässt)
- der Nutzer das zugehörige Formular submitted (vor jedem Submit werden alle Felder automatisch getouched)

SCHEMA-FUNKTION - DISABLED

Inputs können über das Schema disabled werden

```
loginForm = form(this.loginModel, (schemaPath) => {
  email(schemaPath.email);
  required(schemaPath.password);
  disabled(schemaPath.password);
});
```

Felder die disabled sind, werden nicht validiert.

SCHEMA-FUNKTION - DISABLED

Zusätzlich kann der disabled-Funktion eine Bedingung übergeben werden welche festlegt wann das jeweilige Feld disabled sein soll (true = disabled; false = enabled)

```
loginForm = form(this.loginModel, (schemaPath) => {
  email(schemaPath.email);
  disabled(schemaPath.password,
    (ctx) => !ctx.valueOf(schemaPath.email) || ctx.stateOf(schemaPath.email).invalid()
  );
});
```

Der disabled-Funktion übergeben wir dabei an zweiter Stelle eine Arrow-Funktion, diese bekommt den Kontext übergeben. Der Kontext bietet und zwei Funktionen:

- valueOf - returned den Value des übergebenen Pfads
- stateOf - returned den Status des übergebenen Pfads

```
loginForm = form(this.loginModel, (schemaPath) => {
  email(schemaPath.email);
  disabled(schemaPath.password,
    ({valueOf, stateOf}) => !valueOf(schemaPath.email) || stateOf(schemaPath.email).invalid()
  );
});
```

SCHEMA-FUNKTION - DISABLED

Statt true kann auch ein string zurückgegeben werden

```
loginForm = form(this.loginModel, (schemaPath) => {
  email(schemaPath.email);
  disabled(schemaPath.password, ({ valueOf, stateOf }) => {
    if (!valueOf(schemaPath.email)) {
      return 'Email ist nicht gefüllt!';
    } else if (stateOf(schemaPath.email).invalid()) {
      return 'Email ist nicht valide!';
    } else {
      return false;
    }
  });
});
```

Die Gründe warum ein Feld disabled ist können aus dem disabledReasons-Signal gelesen werden

```
@for (reason of loginForm.password().disabledReasons(); track $index) {
  <p>Disabled weil {{ reason.message }}</p>
}
```

Felder die hidden sind werden weiterhin validiert.

SCHEMA-FUNKTION - HIDDEN

Inputs können über das Schema versteckt werden

```
loginForm = form(this.loginModel, (schemaPath) => {
  email(schemaPath.email);
  required(schemaPath.password);
  hidden(schemaPath.password);
});
```

Im Template kann das zugehörige Input mithilfe des hidden-Signal versteckt werden:

```
@if (!loginForm.password().hidden()) {
  <mat-form-field testId="password">
    <mat-label>{{ translate('password') }}</mat-label>
    <input id="password" type="password" matInput [formField]="loginForm.password" />
  </mat-form-field>
}
```

SCHEMA-FUNKTION - HIDDEN

Zusätzlich kann der hidden-Funktion eine Bedingung übergeben werden welche festlegt wann das jeweilige Feld versteckt sein soll (true = hidden; false = visible)

```
loginForm = form(this.loginModel, (schemaPath) => {  
  email(schemaPath.email);  
  hidden(schemaPath.password, ({ valueOf }) => !valueOf(schemaPath.email));  
});
```

SCHEMA-FUNKTION - READONLY

Mithilfe der readonly-Funktion können Felder für Änderungen gesperrt werden:

```
loginForm = form(this.loginModel, (schemaPath) => {  
  email(schemaPath.email);  
  readonly(schemaPath.password);  
});
```

Auch bei readonly ist es möglich eine Bedingung mitzugeben

```
loginForm = form(this.loginModel, (schemaPath) => {  
  email(schemaPath.email);  
  readonly(schemaPath.password, ({ valueOf }) => !valueOf(schemaPath.email));  
});
```

SCHEMA-FUNKTION - HIDDEN, DISABLED, READONLY

Alle drei Funktionen sorgen dafür das entsprechende Felder nicht validiert werden und der Nutzer sie nicht bearbeiten kann.

	hidden()	disabled()	readonly()
Sichtbar für den User	✗	✓	✓
focus/select möglich	✗	✗	✓
Teil der HTML form submission	✗	✗	✓

SCHEMA-FUNKTION - META DATA

Formularfelder können um Metadaten erweitert werden. Einige der built-in Validatoren für Angular tun das out of the box:

```
registerForm = form(this.registerModel, (schemaPath) => {
  required(schemaPath.age);
  min(schemaPath.age, 18);
  max(schemaPath.age, 120);
});

<input type="number" [FormField]="registerForm.age" />
<mat-form-field testId="password">
  <mat-label>Alter</mat-label>
  <mat-hint>Zwischen {{ registerForm.age().min() }} - {{ registerForm.age().max() }}</mat-hint>
  <input type="text" matInput [FormField]="registerForm.age" />
</mat-form-field>
```

SIGNAL-FORMS - EIGENE FORMULARFELD-KOMPONENTEN

Mit Signal-Forms ist es relativ einfach eigene Formularfeld-Komponenten zu entwickeln. Dazu muss die Komponente das FormValueControl-Interface implementieren

```
export class AgeInputComponent implements FormValueControl<number> {
  readonly value = model(0);

  protected inc(): void {
    this.value.update((v) => v + 1);
  }

  protected dec(): void {
    this.value.update((v) => Math.max(v - 1, 0));
  }
}

<div class="age">{{ value() }}</div>
<div>
  <button type="button" (click)="inc()" matIconButton>
    <mat-icon>add</mat-icon>
  </button>
  <button type="button" (click)="dec()" matIconButton>
    <mat-icon>remove</mat-icon>
  </button>
</div>
```

Das Interface erzwingt das wir ein ModelSignal mit dem Namen value erstellen. Das ModelSignal hält den aktuellen Wert des Formularfelds und ermöglicht es diesen zu verändern.

SIGNAL-FORMS - EIGENE FORMULARFELD-KOMPONENTEN

Wir können zusätzlich weitere properties wie disabled oder errors implementieren.

```
export class AgeInputComponent implements FormValueControl<number> {
  readonly value = model(0);
  readonly disabled = input(false);
  ...
}

<div class="age">{{ value() }}</div>
<div>
  <button [disabled]="disabled()" type="button" (click)="inc()" matIconButton>
    <mat-icon>add</mat-icon>
  </button>
  <button [disabled]="disabled()" type="button" (click)="dec()" matIconButton>
    <mat-icon>remove</mat-icon>
  </button>
</div>
```

SIGNAL-FORMS - EIGENE FORMULARFELD-KOMPONENTEN

- Für eigene Checkbox-Komponenten gibt es das Interface `FormCheckboxControl`, dieses erfordert ein `ModelSignal` mit dem Namen `checked`
- Die Erstellung einer eigenen Formularfeld-Komponente ist unter Signals-Forms wesentlich einfacher als es bei den `ReactiveForms` mit dem `ControlValueAccessor` war.
- Signal-Forms sind abwärtskompatibel zum `ControlValueAccessor`. D.h. wir können bestehende Formularfeld-Komponenten mit `ControlValueAccessor` auch mit Signal-Forms verwenden

SIGNAL-FORMS - ABWÄRTSKOMPATIBILITÄT

Signal-Forms sind Abwärtskompatibel zu `ReactiveForms`. So ist es z.B. möglich einzelne `FormControls` und auch komplette `FormGroups` in Signal-Forms zu integrieren. Mehr dazu => <https://angular.dev/guide/forms/signals/migration>

SIGNAL-FORMS - UNDEFINED

Signalforms erlauben `undefined` nicht als Value:

```
interface Registration {
  firstname: string,
  lastname: string,
  age?: number
}
...
registerModel = {
  firstname: '',
  lastname: '',
  age: undefined
}

registerForm = form(registerModel); // kennt nur die Felder firstname und lastname
```

Das liegt darin begründet das für die Signal Form ein Attribut mit dem Wert `undefined` nicht existiert.

SIGNAL-FORMS - UNDEFINED

Für Fälle in denen einzelne Felder optional sein können, muss daher auf null zurückgegriffen werden

```
interface Registration {  
  firstname: string,  
  lastname: string,  
  age: number | null  
}  
...  
registerModel = {  
  firstname: '',  
  lastname: '',  
  age: null  
}  
  
registerForm = form(registerModel); // kennt die Felder firstname, lastname und age
```

Das liegt darin begründet das für die Signal Form ein Attribut mit dem Wert undefined nicht existiert.

SIGNAL-FORMS

- Signal-Forms wird sehr wahrscheinlich der neue Goldstandard für Formulare in Angular
- Für produktive Anwendungen sollte es, solange es noch ein experimentelles Feature ist, nicht verwendet werden
- Für Anwendungen die in absehbarer Zeit nicht produktiv gehen, wäre der Einsatz eine Überlegung wert
- Sobald Signal-Forms stable sind ist der Umstieg empfehlenswert => Aufgrund der Abwärtskompatibilität muss das nicht als Big-Bang passieren, eine graduelle Migration ist möglich